Labsheet 6

1.Write a C++ program to implement a Queue using an array. Perform Enqueue, Dequeue, Peek and Display operations.

```
int main61() {
    SimpleQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();
    q.dequeue();
    q.peek();

    return 0;
}
```

2. Write a C++ program to implement Enqueue operation and demonstrate insertion at the Rear.

```
int main62() {
    SimpleQueue q;

    q.enqueue(10);
    q.enqueue(20);

    cout << "Queue after insertion: ";
    q.display();

    return 0;
}
```

3. Write a C++ program to implement Dequeue operation and demonstrate deletion from the Front.

```
int main63() {
    SimpleQueue q;

    q.enqueue(10);
    q.enqueue(20);

    q.dequeue();
    q.display();

    return 0;
}
```

4. Write a C++ program to implement Peek operation and display the Front element without deleting it.

```
int main64() {
    SimpleQueue q;

    q.enqueue(10);
    q.enqueue(20);

    q.peek();

    return 0;
}
```

5. Write a C++ program to display all elements of a Queue from Front to Rear.

```
int main65() {
    SimpleQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();

    return 0;
}
```

6. Write a C++ program to demonstrate Queue Overflow using an array.

```
int main66() {
    SimpleQueue q;

    for (int i = 1; i <= 6; i++)
        q.enqueue(i * 10);

    return 0;
}
```

7. Write a C++ program to demonstrate Queue Underflow.

```
int main67() {
    SimpleQueue q;

    q.dequeue();

    return 0;
}
```

8. Write a menu-driven C++ program implementing all basic Queue operations.

```cpp
int main68() {
    SimpleQueue q;
    int choice, value;

    do {
        cout << "\n1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value: ";
                cin >> value;
                q.enqueue(value);
                break;

            case 2:
                q.dequeue();
                break;

            case 3:
                q.peek();
                break;

            case 4:
                q.display();
                break;

            case 5:
                break;

            default:
                cout << "Invalid choice\n";
        }
```

```cpp
    } while (choice != 5);

    return 0;
}
```

9. Write a C++ program to count the number of elements currently present in a Queue.

```cpp
int main69() {
    SimpleQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Number of elements = " << q.count();

    return 0;
}
```

10. Write a C++ program to search for a given element in a Queue.

```cpp
int main70() {
    SimpleQueue q;
    int key;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Enter key: ";
    cin >> key;

    q.search(key);

    return 0;
}
```

11. Write a C++ program to find the maximum and minimum elements in a Queue.

```cpp
int main71() {
    SimpleQueue q;

    q.enqueue(30);
    q.enqueue(10);
    q.enqueue(50);
    q.enqueue(20);

    q.maxMin();

    return 0;
}
```

12. Write a C++ program to reverse the elements of a Queue.

```cpp
int main72() {
    int q[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> q[i];

    for (int i = 0; i < n / 2; i++)
        swap(q[i], q[n - 1 - i]);

    cout << "Reversed queue: ";
    for (int i = 0; i < n; i++)
        cout << q[i] << " ";

    return 0;
}
```

13. Write a C++ program to implement a Circular Queue using an array.

```cpp
class CircularQueue {
    int q[5];
    int front;
    int rear;

public:
    CircularQueue() {
        front = -1;
        rear = -1;
    }

    void enqueue(int value) {
        if ((rear + 1) % 5 == front) {
            cout << "Circular Queue Overflow\n";
            return;
        }

        if (front == -1)
            front = 0;

        rear = (rear + 1) % 5;
        q[rear] = value;
    }

    void dequeue() {
        if (front == -1) {
            cout << "Circular Queue Underflow\n";
            return;
        }

        cout << q[front] << " deleted\n";

        if (front == rear) {
            front = rear = -1;
        }
        else {
            front = (front + 1) % 5;
        }
    }

    void display() {
        if (front == -1) {
            cout << "Empty\n";
            return;
        }

        int i = front;

        while (true) {
            cout << q[i] << " ";

            if (i == rear)
                break;

            i = (i + 1) % 5;
        }

        cout << endl;
    }
};

int main73() {
    CircularQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();
    q.dequeue();
    q.enqueue(40);

    q.display();

    return 0;
}
```

14. Write a C++ program to implement Queue using a Linked List.

```
struct QNode {
    int data;
    QNode* next;

    QNode(int value) {
        data = value;
        next = NULL;
    }
};

int main74() {
    QNode* frontQ = NULL;
    QNode* rearQ = NULL;

    QNode* newNode = new QNode(10);
    frontQ = rearQ = newNode;

    newNode = new QNode(20);
    rearQ->next = newNode;
    rearQ = newNode;

    cout << "Queue: ";
    QNode* temp = frontQ;

    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }

    return 0;
}
```

15. Write a C++ program to compare Linear Queue and Circular Queue in terms of space utilization and operations.

```
int main75() {
    cout << "Linear Queue: simple implementation, but unused front spaces may be wasted.\n";
    cout << "Circular Queue: reuses empty spaces using circular indexing.\n";
    cout << "Enqueue: O(1)\n";
    cout << "Dequeue: O(1)\n";
    cout << "Display: O(n)\n";

    return 0;
}
```